DuocUC® INFORMÁTICA Y TELECOMUNICACIONES



INFORME DE EVIDENCIAS – PARCIAL No. 2

Integrantes:

Araneda, Benjamin

Villalobos, Vicente

Vitta, Dylan

Sección: TCY0101 – 001

Profesor: Alicia Ramos

Nombre de la Asignatura:

TCY0101 – TÉCNICAS DE CALIDAD DE SOFTWARE

Resumen de Resultados y Métricas
Sistema: SmartLogix
Realizado por: Benjamin Araneda, Vicente Villalobos y Dylan Vitta

Consolidado de todas las pruebas ejecutadas sobre SmartLogix. En total se ejecutaron 701 casos de prueba (funcionales, unitarios y de integración), con 698 aprobados (99,6 %), además de análisis estático, escaneos de vulnerabilidades y pruebas de rendimiento con dos herramientas. El detalle de cada prueba está en las hojas de evidencia de las secciones siguientes.

1. Casos de prueba ejecutados

Tipo de prueba	Herramienta	Ejecutadas	Aprobadas	Fallidas	% éxito	Estatus (hoja)
Funcionales automatizadas	Katalon Recorder 7.1	16	16	0	100 %	Aprobada (PA-01)
Unitarias	JUnit 5 + Mockito	681	678	3	99,6 %	Rechazada (PU-01)
Integración	JUnit 5 + Spring Boot Test	4	4	0	100 %	Aprobada (PI-01)
TOTAL casos de prueba		**701**	**698**	**3**	**99,6 %**	

2. Métricas de calidad, seguridad y rendimiento

Tipo de prueba	Herramienta	Métricas obtenidas
Inspección de código (backend)	SonarQube Cloud	0 bugs, 13 vulnerabilidades (11 revisadas como seguras en contexto), 184 code smells, 0 % duplicación, cobertura 90,6 % en Orden. Diferida (PS-01).
Inspección de código (frontend)	SonarQube Cloud	Quality Gate aprobado; 48 bugs (accesibilidad), 3 vulnerabilidades, 75 code smells, 2,3 % duplicación. Rechazada (PS-02).
Vulnerabilidades (frontend)	OWASP ZAP 2.17	13 endpoints: 0 Altas, 3 Medias, 2 Bajas, 2 Informativas. Rechazada (PV-01).
Vulnerabilidades (APIs)	OWASP ZAP 2.17	142 endpoints de 6 microservicios: 0 Altas, 0 Medias, 3 Bajas, 2 Informativas. Aprobada (PV-02).
Carga normal	Apache JMeter / k6	JMeter: 1.045 req, 0 % errores, p95 15 ms. k6: 1.782 req, 0 % errores, p95 18,73 ms. Aprobadas (PC-01, PC-02).
Estrés de lectura (150 usuarios)	Apache JMeter / k6	JMeter: 42.700 req, 0 % errores, 285 req/s promedio, pico 489 req/s, p95 17 ms. k6: 31.174 req, 0 % errores, p95 13,59 ms. Aprobadas (PE-01, PE-02).
Estrés de escritura (75 usuarios)	k6	6.226 req, 0 errores 5xx, 0 timeouts, p95 117,82 ms; stock final 0 sin sobreventa. Aprobada (PE-03).

3. Defectos y hallazgos consolidados

N°	Hallazgo	Severidad	Origen	Estado
1	/auth/cambiar-clave responde 403 en vez de 400 ante una clave corta.	Media	PU-01 (JUnit)	Abierto
2	Respuestas HTTP 500 con detalle de error ante entradas inválidas en Rol, Users, Inventario, Producto y Estado.	Media	PV-02 (ZAP)	Abierto

N°	Hallazgo	Severidad	Origen	Estado
3	Frontend sin Content-Security-Policy ni protección anti-clickjacking.	Media	PV-01 (ZAP)	Abierto
4	/actuator/health accesible a través del proxy del frontend.	Media	PV-01 (ZAP)	Abierto
5	41 campos de formulario sin etiqueta asociada (accesibilidad).	Media	PS-02 (SonarQube)	Abierto
6	Dos pruebas unitarias desactualizadas frente al código (BodegaServiceTest, SecurityConfigTest de Users).	Baja	PU-01 (JUnit)	Abierto
7	Endpoints de FallbackController sin método HTTP explícito (6).	Baja	PS-01 (SonarQube)	Abierto
8	Correo electrónico enviado en query string (/api/roles/assign?email=).	Baja	PV-02 (ZAP)	Abierto
9	El servidor revela “X-Powered-By: Express” y falta X-Content-Type-Options.	Baja	PV-01 (ZAP)	Abierto

Ejecución de Pruebas Automatizadas

Sistema: SmartLogix

Realizado por: Benjamin Araneda, Vicente Villalobos y Dylan Vitta

<table>
<tr><th colspan="3">EVIDENCIA DE EJECUCIÓN DE PRUEBAS: AUTOMATIZADAS</th></tr>
<tr><td>ID. Prueba:
PA-01 – Suite Katalon “SmartLogix_completo” (16 casos de prueba)</td><td>Id. Caso de Uso Asociado:
CU-01 Iniciar Sesión; CU-02 Gestión de Usuarios (roles); CU-03 Gestión de Productos; CU-05 Gestión de Órdenes; CU-09 Cierre de Sesión</td><td>Id. Del Requerimiento Asociado:
RF-01, RF-02, RF-03, RF-05, RF-07, RNF-05</td></tr>
<tr><td>Fecha de Ejecución:
22/10/2026 20:49</td><td>Herramienta(s) Ocupada(s):
Katalon Recorder 7.1.0 (extensión de Google Chrome basada en el motor de Selenium IDE; comandos Selenese)</td><td>Alcance:
Pruebas funcionales end-to-end sobre la interfaz web: autenticación, sesión, control de acceso por rol, carrito y ciclo de pedidos.</td></tr>
<tr><td>Ambiente de Prueba:</td><td colspan="2">Local – Windows 11 Pro. Stack completo levantado con docker-compose: Gateway/BFF (:80), 8 microservicios Spring Boot (Users :8082, Rol :8081, Estado :8086, Inventario :8083, Producto :8085, Orden :8084, Configuración :8087), PostgreSQL y Kafka. Frontend Angular en http://localhost:4200. Usuarios de prueba: admin@smartlogix.cl y cliente@smartlogix.cl.</td></tr>
<tr><td>Resultado Esperado:</td><td colspan="2">– Autenticación (Login, LoginFallido, LoginCamposVacios, LoginCorreoInvalido): con credenciales válidas se entra al Dashboard; con clave incorrecta aparece “Credenciales inválidas. Verifica tu correo y contraseña.”; con campos vacíos o un correo mal formado (“Correo inválido.”) el botón “Iniciar sesión” queda deshabilitado.
– Sesión (UrlDashboard, AccesoSinSesion, CerrarSesion): tras el login la URL es /dashboard; sin sesión, /dashboard y /ordenes redirigen a /login; “Cerrar sesión” borra la sesión y no permite volver al Dashboard.
– Control de acceso por rol (MenuCliente, MenuAdmin, AdminProductos, ClienteSinAccesoProductos): el cliente ve solo Dashboard, Órdenes y Mi Perfil; el administrador ve Productos, Inventario, Órdenes, Envíos y Usuarios y abre el</td></tr>
</table>

EVIDENCIA DE EJECUCIÓN DE PRUEBAS: **AUTOMATIZADAS**	
	Catálogo de Productos; si el cliente entra a /productos, vuelve al Dashboard. – **Pedidos (CREARORDEN, CarritoVacio, CarritoAgregarQuitar, CancelarPedido, FiltroPendiente):** crear un pedido aumenta en 1 el total y deja una fila "Pendiente" con el precio del producto; con el carrito vacío no se puede confirmar; + y − ajustan la cantidad sin crear pedidos; cancelar un pedido lo deja "Cancelado" con el aviso "Orden cancelada"; el filtro "Pendiente" oculta los pedidos de otros estados.
Resultado Obtenido:	Ejecución completa de la suite: 16 de 16 casos aprobados, todos en verde y sin fallos de verificación. El log termina con "Test case passed" (22-10-2026 20:49). – CREARORDEN validó con verifyText el nuevo total ("N+1 pedidos registradas"), la fila nueva en estado "Pendiente" y el total igual al precio del producto. – CancelarPedido confirmó el cambio a "Cancelado", el aviso "Orden cancelada" y la desaparición del botón Cancelar; luego FiltroPendiente verificó que esa orden queda oculta. – Los 4 casos de control de acceso confirmaron que cada rol ve solo sus menús y que una URL no permitida redirige al Dashboard. La suite reúne 16 casos de prueba automatizados, por sobre el mínimo de 15 exigido.
Observaciones:	Durante la automatización se detectaron y corrigieron problemas que generaban falsos positivos en los scripts originales: – el comando submit (css=.space-y-4) quedaba en verde pero no autenticaba, porque Angular no recibe el evento de form.submit(). Se reemplazó por click en el botón "Iniciar sesión" con un XPath exacto (existe otro botón "Iniciar sesión con AWS Cognito"). – la sesión persiste en localStorage y /login redirige a /dashboard; cada caso limpia la sesión con runScript al inicio para que sea independiente. – Katalon Recorder es una evolución de Selenium IDE: usa su mismo motor (Selenium Core) y sus comandos Selenese (open, click, type, verifyText, waitForElementPresent, etc.), por lo que los casos son equivalentes a los de Selenium IDE.

<table>
<tr><th colspan="3">EVIDENCIA DE EJECUCIÓN DE PRUEBAS: AUTOMATIZADAS</th></tr>
<tr><td></td><td colspan="2">La versión 7.1 no soporta executeScript al importar HTML; se usó runScript.
– el Gateway limita a 5 intentos de login por minuto por IP (LoginRateLimitFilter). Es un control de seguridad que funciona según lo diseñado; cada caso espera 12,5 s antes de iniciar sesión, por lo que la suite completa tarda unos 4 minutos.
– la lista de pedidos no está ordenada por ID, por eso la verificación compara el total y el ID nuevo en lugar de la primera fila.
– CREARORDEN crea un pedido real y CancelarPedido cancela uno en la cuenta del cliente de prueba; la suite debe ejecutarse completa y en orden.</td></tr>
<tr><td>Realizado Por:
Benjamin Araneda, Vicente Villalobos y Dylan Vitta</td><td>Autorizada Por:
Pendiente – revisión docente</td><td>Estatus: Marque una Equis
[X] Aprobada
[ ] Rechazada
[ ] Diferida</td></tr>
</table>

EVIDENCIA DE EJECUCIÓN DE PRUEBAS: AUTOMATIZADAS

Evidencia de ejecución:

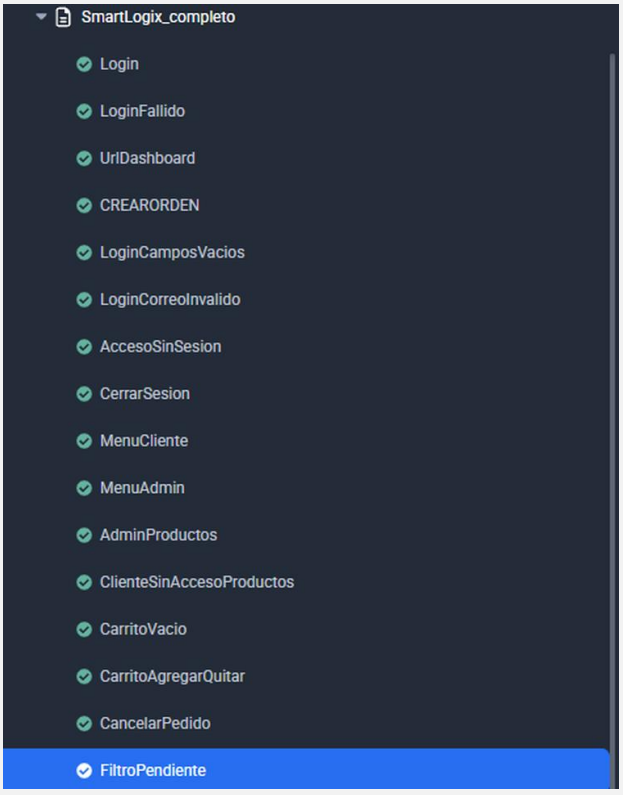


Figura PA-01a. Suite SmartLogix_completo en Katalon Recorder: los 16 casos aprobados (detalle ampliado).

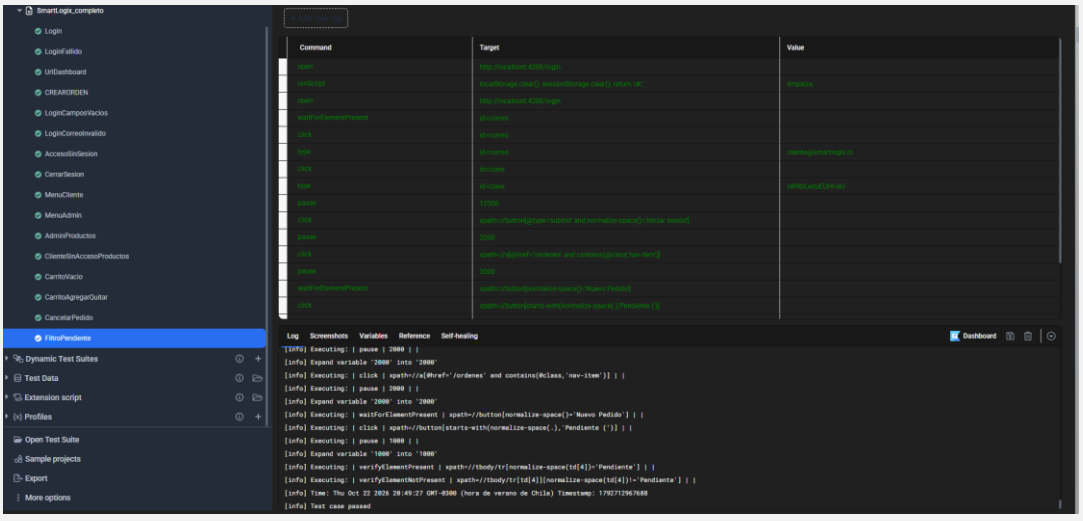


Figura PA-01b. Katalon Recorder 7.1.0: comandos del último caso y log de ejecución finalizado con "Test case passed".

Ejecución de Pruebas Unitarias y de Integración

Sistema: SmartLogix

Realizado por: Benjamin Araneda, Vicente Villalobos y Dylan Vitta

<table>
<tr><th colspan="3">EVIDENCIA DE EJECUCIÓN DE PRUEBAS: UNITARIAS Y DE INTEGRACIÓN</th></tr>
<tr><td>ID. Prueba:
PU-01 – Pruebas unitarias JUnit 5 de los 8 microservicios (681 pruebas)</td><td>Id. Caso de Uso Asociado:
CU-01, CU-02, CU-03, CU-04, CU-05, CU-07</td><td>Id. Del Requerimiento Asociado:
RF-01 a RF-07, RF-09; RNF-03, RNF-05</td></tr>
<tr><td>Fecha de Ejecución:
22/10/2026</td><td>Herramienta(s) Ocupada(s):
JUnit 5, Mockito, Spring Boot Test, Maven Surefire 3, JaCoCo, Java 21</td><td>Alcance:
Capa de servicio, controladores, clientes HTTP, seguridad (JWT, filtros) y productores/consumidores Kafka de cada microservicio, aislados con mocks.</td></tr>
<tr><td>Ambiente de Prueba:</td><td colspan="2">Local – Windows 11 Pro, JDK 21, Maven 3.9.11. Ejecución con “mvn test” en cada microservicio; dependencias externas simuladas con Mockito.</td></tr>
<tr><td>Resultado Esperado:</td><td colspan="2">Todas las pruebas unitarias se ejecutan sin fallos ni errores en los 8 microservicios.</td></tr>
<tr><td>Resultado Obtenido:</td><td colspan="2">681 pruebas ejecutadas: 678 aprobadas, 1 fallo y 2 errores, 0 omitidas. 6 servicios con BUILD SUCCESS; Inventario y Users con BUILD FAILURE. Resumen de los reportes Surefire (cobertura JaCoCo solo donde el reporte se regeneró en esta ejecución):
<table>
<tr><th>Servicio</th><th>Clases</th><th>Tests</th><th>Fallos</th><th>Errores</th><th>Omit.</th><th>Cob. líneas</th></tr>
<tr><td>Configuracion</td><td>4</td><td>23</td><td>0</td><td>0</td><td>0</td><td>–</td></tr>
<tr><td>Estado</td><td>11</td><td>65</td><td>0</td><td>0</td><td>0</td><td>73.3 %</td></tr>
<tr><td>Gateway</td><td>5</td><td>33</td><td>0</td><td>0</td><td>0</td><td>44.3 %</td></tr>
<tr><td>Inventario</td><td>16</td><td>130</td><td>0</td><td>1</td><td>0</td><td>–</td></tr>
<tr><td>Orden</td><td>13</td><td>103</td><td>0</td><td>0</td><td>0</td><td>80.4 %</td></tr>
<tr><td>Producto</td><td>9</td><td>68</td><td>0</td><td>0</td><td>0</td><td>49.9 %</td></tr>
<tr><td>Rol</td><td>16</td><td>130</td><td>0</td><td>0</td><td>0</td><td>77.8 %</td></tr>
<tr><td>Users</td><td>17</td><td>129</td><td>1</td><td>1</td><td>0</td><td>–</td></tr>
<tr><td>TOTAL</td><td>91</td><td>681</td><td>1</td><td>2</td><td>0</td><td></td></tr>
</table></td></tr>
<tr><td>Observaciones:</td><td colspan="2">Análisis de las 3 pruebas que no pasaron:
– Inventario – BodegaServiceTest.eliminar_existente_debeDesactivarBodegaYPublicarEvento: NullPointerException porque el servicio ahora usa EstanteRepository</td></tr>
</table>

<table>
<tr><th colspan="3">EVIDENCIA DE EJECUCIÓN DE PRUEBAS: UNITARIAS Y DE INTEGRACIÓN</th></tr>
<tr><td></td><td colspan="2">(desactivación en cascada de estantes) y la prueba no lo simula. Es una prueba desactualizada: falta agregar @Mock EstanteRepository.
– Users – SecurityConfigTest.corsConfigurationSource_retornaConfiguracion: allowedOrigins se inyecta con @Value y la prueba unitaria instancia SecurityConfig sin asignarlo. Es un defecto de la prueba: usar ReflectionTestUtils.setField o un contexto de Spring.
– Users – AuthControllerTest.cambiarClave_claveCorta_retorna400: se esperaba 400 y se obtuvo 200. El controlador no valida el largo de la clave y delega en RecuperacionService (simulado en la prueba). En el sistema real, una clave corta es rechazada por UserService.validarContraseña, pero el error se responde con 403 en lugar de 400. Es un defecto real del código de estado HTTP en /auth/cambiar-clave.
Las ejecuciones anteriores (mayo a octubre) estaban todas en verde; los fallos aparecieron al volver a ejecutar sobre el código actual, lo que muestra la importancia de correr la suite completa antes de cada entrega.</td></tr>
<tr><td>Realizado Por:
Benjamin Araneda, Vicente Villalobos y Dylan Vitta</td><td>Autorizada Por:
Pendiente – revisión docente</td><td>Estatus: Marque una Equis
[ ] Aprobada
[X] Rechazada
[ ] Diferida</td></tr>
</table>

EVIDENCIA DE EJECUCIÓN DE PRUEBAS: UNITARIAS Y DE INTEGRACIÓN

Evidencia de ejecución:

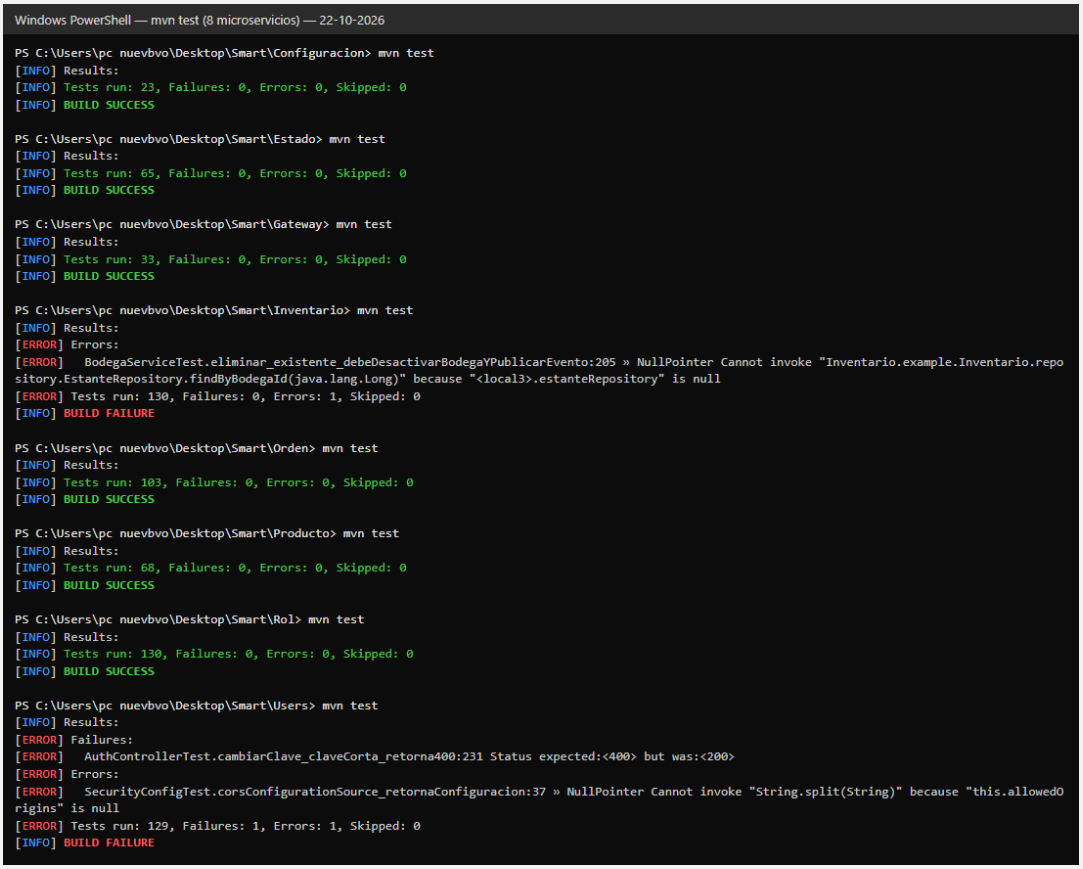


Figura PU-01. Resultado de "mvn test" en los 8 microservicios (extracto de la consola, 22-10-2026).

EVIDENCIA DE EJECUCIÓN DE PRUEBAS: **UNITARIAS Y DE INTEGRACIÓN**		
ID. Prueba: PI-01 – OrdenIntegrationTest (creación de orden end-to-end en el microservicio Orden)	**Id. Caso de Uso Asociado:** CU-05 Gestión de Órdenes	**Id. Del Requerimiento Asociado:** RF-02, RF-04, RF-05, RNF-05
Fecha de Ejecución: 22/10/2026	**Herramienta(s) Ocupada(s):** JUnit 5, @SpringBootTest, MockMvc, H2 en memoria, @MockitoBean, Maven Surefire	**Alcance:** Integración real de controller, cadena de filtros de seguridad, servicio y repositorio JPA del microservicio Orden; solo se simulan Users, Producto y el productor Kafka.
Ambiente de Prueba:	Contexto Spring completo con perfil “test” (application-test.properties) y base H2 en memoria.	
Resultado Esperado:	– POST /api/ordenes sin autenticación responde 403. – como cliente: responde 200, la orden queda en estado “pendiente” con el detalle del producto, aparece en GET /api/ordenes/mis-ordenes y se publica el evento “orden creada”. – con un rol sin permiso (transportista) responde 403. – sin stock disponible responde 400 con el mensaje “Stock insuficiente”.	
Resultado Obtenido:	4 pruebas ejecutadas, 0 fallos, 0 errores, en 14,18 s (incluye el arranque del contexto Spring en 12,6 s), con Java 21.0.6 y el perfil “test”.	
Observaciones:	La prueba verifica la persistencia leyendo la orden desde el listado del cliente, no solo el código 200 del POST.	
Realizado Por: Benjamin Araneda, Vicente Villalobos y Dylan Vitta	**Autorizada Por:** Pendiente – revisión docente	**Estatus:** Marque una Equis **[X] Aprobada** [ ] Rechazada [ ] Diferida

EVIDENCIA DE EJECUCIÓN DE PRUEBAS: UNITARIAS Y DE INTEGRACIÓN

Evidencia de ejecución:

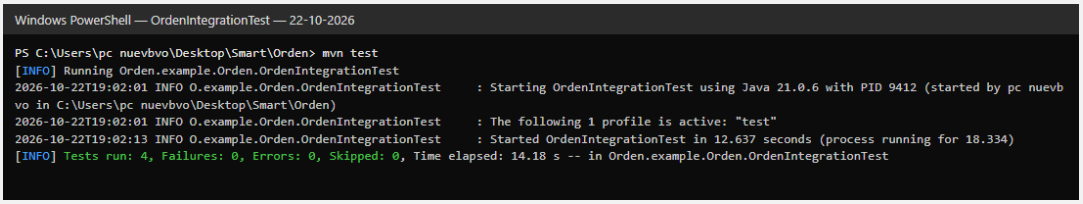


Figura PI-01. Ejecución de OrdenIntegrationTest: 4 pruebas, 0 fallos (consola, 22-10-2026).

Inspección de Código Estático

Sistema: SmartLogix

Realizado por: Benjamin Araneda, Vicente Villalobos y Dylan Vitta

<table>
<tr><th colspan="3">EVIDENCIA DE EJECUCIÓN DE PRUEBAS DE: INSPECCIÓN DE CÓDIGO ESTÁTICO</th></tr>
<tr><td>ID. Prueba:
PS-01 – Análisis estático del backend (BFF, Orden y Producto)</td><td>Id. Caso de Uso Asociado:
CU-01, CU-03, CU-05</td><td>Id. Del Requerimiento Asociado:
RNF-03, RNF-05, RNF-07</td></tr>
<tr><td>Fecha de Ejecución:
25/09/2026 (último análisis); Orden.example:Orden 04/06/2026</td><td>Herramienta(s) Ocupada(s):
SonarQube Cloud (SonarCloud, organización benjazzx), SonarScanner para Maven, JaCoCo</td><td>Alcance:
Código Java de los proyectos SmartLogix_BFF (Gateway), Orden_SmartLogix, Producto_SmartLogix y Orden.example:Orden.</td></tr>
<tr><td>Ambiente de Prueba:</td><td colspan="2">Análisis en SonarCloud a partir del código fuente y binarios compilados (sonar-project.properties de cada servicio).</td></tr>
<tr><td>Resultado Esperado:</td><td colspan="2">Quality Gate aprobado, 0 bugs, 0 vulnerabilidades sin revisar y calificaciones A en confiabilidad, seguridad y mantenibilidad.</td></tr>
<tr><td>Resultado Obtenido:</td><td colspan="2">
<pre>
Proyecto   Bugs Vuln. Smells Dupl.   Cob.   LOC   Conf/Seg/Mant
----
SmartLogix_BFF   0   9   20   0,0%   –   845 A / D / A
Orden_SmartLogix   0   2   57   0,0%   – 1723 A / D / A
Producto_SmartLogix   0   2   40   0,0%   – 2064 A / D / A
Orden.example:Orden   0   0   67   0,0% 90,6% 1249 A / A / A
</pre>
Quality Gate: aprobado en Orden.example:Orden; “no calculado” en SmartLogix_BFF, Orden_SmartLogix y Producto_SmartLogix porque no tienen código nuevo suficiente para evaluarlo.
Nota: los conteos de la tabla corresponden a las métricas clásicas (bugs, vulnerabilidades, code smells). El panel actual de SonarCloud agrupa los issues por calidad de software, por eso las capturas muestran otros conteos (por ejemplo, 9 issues de seguridad en BFF).
</td></tr>
<tr><td>Hallazgos Detectados:</td><td colspan="2">– 6 × java:S4502 (Crítica): Spring Security con CSRF deshabilitado en SecurityConfig de Orden, Producto y BFF (incluye CognitoSecurityConfig). Revisado: son APIs stateless con JWT y sin cookies de sesión, por lo que se considera seguro y debe marcarse como “Safe” en SonarCloud.</td></tr>
</table>

<table>
<tr><th colspan="3">EVIDENCIA DE EJECUCIÓN DE PRUEBAS DE: INSPECCIÓN DE CÓDIGO ESTÁTICO</th></tr>
<tr><td></td><td colspan="2">– 2 × java:S2077 (Mayor): CREATE DATABASE construido por concatenación en OrdenApplication y ProductoApplication. El nombre viene de una variable de entorno y no de la entrada del usuario (riesgo bajo); se recomienda validarlo con una expresión regular.
– 6 × java:S3752 (Menor): endpoints de FallbackController sin método HTTP explícito. Corrección: usar @GetMapping o @RequestMapping(method = …).
0 bugs y 0 % de duplicación en los 4 proyectos; deuda técnica con calificación A.</td></tr>
<tr><td>Realizado Por:
Benjamin Araneda, Vicente Villalobos y Dylan Vitta</td><td>Autorizada Por:
Pendiente – revisión docente</td><td>Estatus: Marque una Equis
[ ] Aprobada
[ ] Rechazada
[X] Diferida</td></tr>
</table>

Evidencia de ejecución:

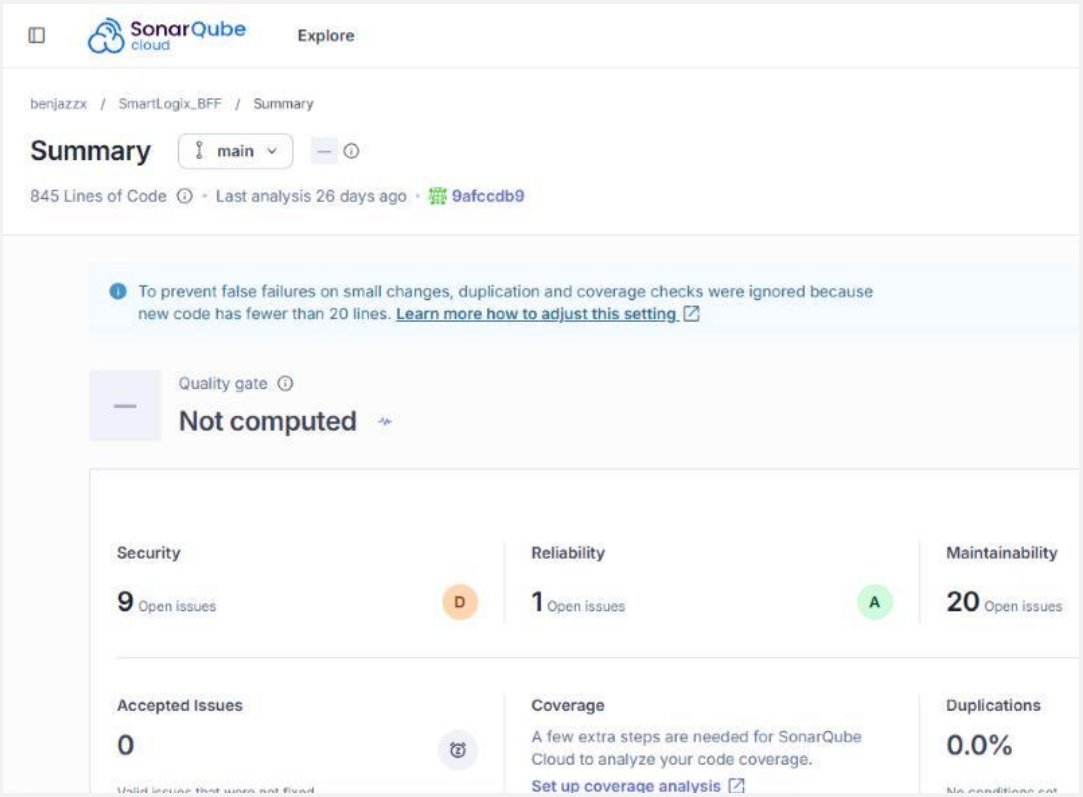


Figura PS-01a. Resumen de SonarCloud del proyecto SmartLogix_BFF.

EVIDENCIA DE EJECUCIÓN DE PRUEBAS DE: INSPECCIÓN DE CÓDIGO ESTÁTICO

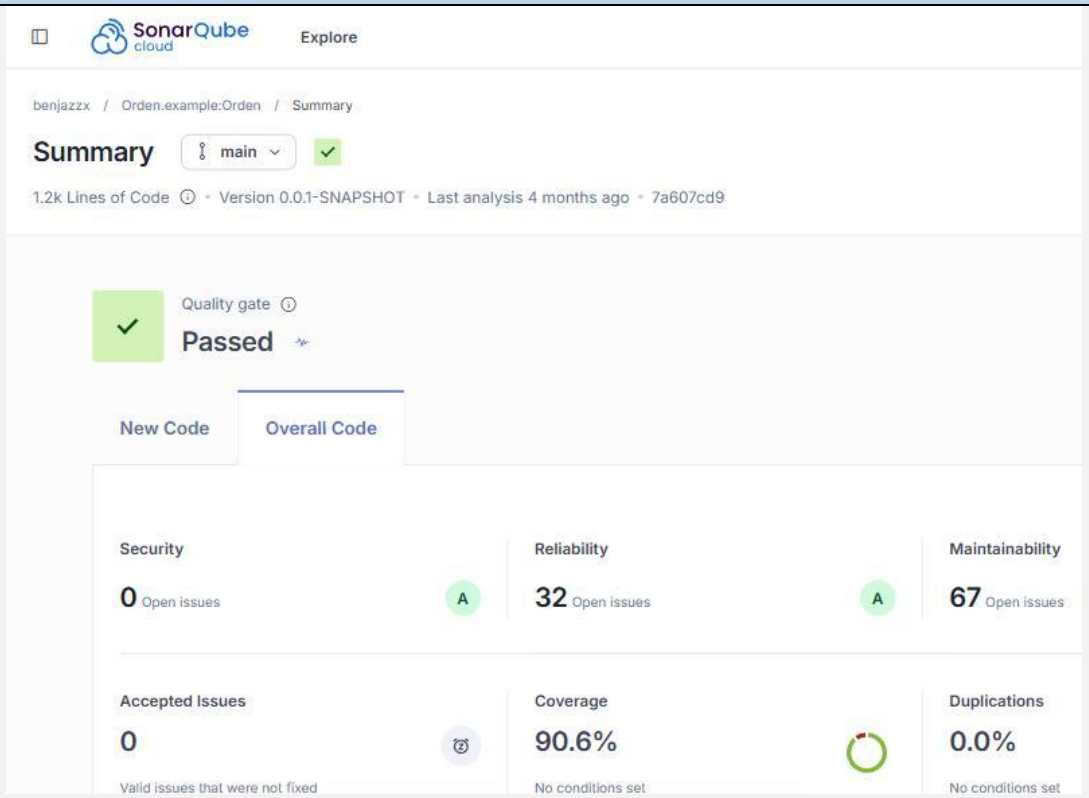


Figura PS-01b. Resumen de SonarCloud de Orden.example:Orden (Quality Gate aprobado, cobertura 90,6 %).

EVIDENCIA DE EJECUCIÓN DE PRUEBAS DE: **INSPECCIÓN DE CÓDIGO ESTÁTICO**		
ID. Prueba: PS-02 – Análisis estático del frontend Angular (Frontend_Smartlogix)	**Id. Caso de Uso Asociado:** CU-01, CU-05	**Id. Del Requerimiento Asociado:** RNF-06, RNF-10
Fecha de Ejecución: 25/09/2026	**Herramienta(s) Ocupada(s):** SonarQube Cloud (SonarCloud), analizadores TypeScript, HTML, Docker y Kubernetes	**Alcance:** Código del frontend Angular (TypeScript y plantillas HTML), Dockerfile y manifiestos k8s (8.002 líneas).
Ambiente de Prueba:	Análisis en SonarCloud del repositorio Frontend_Smartlogix.	
Resultado Esperado:	Quality Gate aprobado, sin bugs ni vulnerabilidades, calificaciones A.	
Resultado Obtenido:	Rama analizada: feature/recuperacion-cuenta (rama principal del proyecto en SonarCloud). Quality Gate: aprobado. 48 bugs (confiabilidad D), 3 vulnerabilidades (seguridad C), 75 code smells (mantenibilidad A), 2,3 % de duplicación. No se cumple el criterio de 0 bugs ni el de calificaciones A.	
Hallazgos Detectados:	– **41 × Web:InputWithoutLabelCheck:** campos input/select/textarea sin etiqueta asociada (accesibilidad). – **5 × Web:MouseEventWithoutKeyboardEquivalentCheck:** eventos de mouse sin equivalente de teclado. – **typescript:S2871 y Web:S5256:** sort() sin función de comparación (1) y tabla sin encabezados (1). – **2 × docker:S6505 (vulnerabilidad):** npm install sin "--ignore-scripts" en el Dockerfile (líneas 9 y 22). – **1 × kubernetes:S6865 (vulnerabilidad):** service account montado automáticamente en k8s/frontend.yaml.	
Realizado Por: Benjamin Araneda, Vicente Villalobos y Dylan Vitta	**Autorizada Por:** Pendiente – revisión docente	**Estatus:** Marque una Equis [ ] Aprobada **[X] Rechazada** [ ] Diferida

EVIDENCIA DE EJECUCIÓN DE PRUEBAS DE: INSPECCIÓN DE CÓDIGO ESTÁTICO

Evidencia de ejecución:

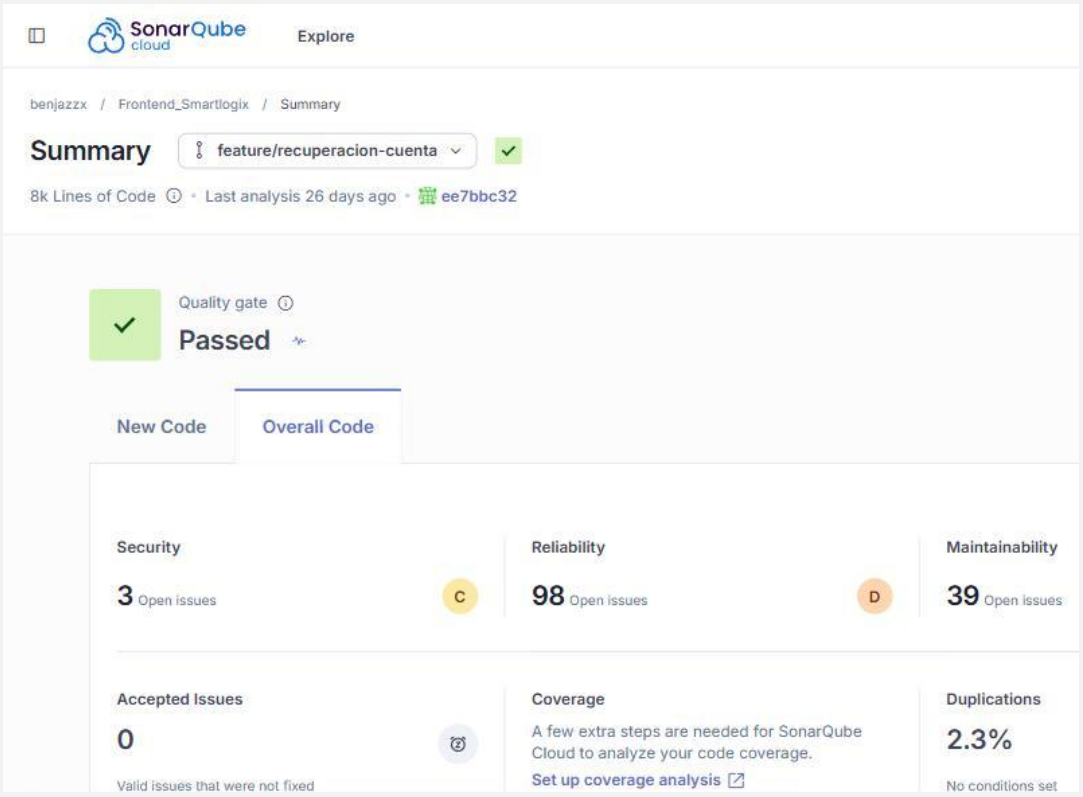


Figura PS-02. Resumen de SonarCloud del proyecto Frontend_Smartlogix.

Ejecución de Pruebas de Vulnerabilidad

Sistema: SmartLogix

Realizado por: Benjamin Araneda, Vicente Villalobos y Dylan Vitta

EVIDENCIA DE EJECUCIÓN DE PRUEBAS DE: VULNERABILIDAD		
ID. Prueba: PV-01 – Escaneo OWASP ZAP del frontend (http://localhost:4200)	**Id. Caso de Uso Asociado:** CU-01 Iniciar Sesión; CU-05 Gestión de Órdenes	**Id. Del Requerimiento Asociado:** RNF-04, RNF-05
Fecha de Ejecución: 22/10/2026 14:26	**Herramienta(s) Ocupada(s):** OWASP ZAP 2.17.0 (spider + escaneo pasivo y activo)	**Alcance:** Aplicación Angular servida en localhost:4200 (13 endpoints: HTML, JS, CSS y el proxy a /actuator).
Ambiente de Prueba:	Local – Windows 11 Pro. Stack completo levantado con docker-compose: Gateway/BFF (:80), 8 microservicios Spring Boot (Users :8082, Rol :8081, Estado :8086, Inventario :8083, Producto :8085, Orden :8084, Configuración :8087), PostgreSQL y Kafka. Frontend Angular en http://localhost:4200. Frontend servido por el servidor de desarrollo de Angular (Express).	
Resultado Esperado:	0 alertas de riesgo Alto y 0 de riesgo Medio; cabeceras de seguridad HTTP presentes.	
Resultado Obtenido:	0 Altas, 3 Medias, 2 Bajas y 2 Informativas. No se cumple el criterio de 0 alertas Medias.	
Vulnerabilidades Detectadas:	– **Media – CSP Header Not Set:** falta la cabecera Content-Security-Policy (3 instancias: /, /dashboard). Mitigación: definir una CSP en el servidor que publica el frontend. – **Media – Missing Anti-clickjacking Header:** sin X-Frame-Options ni frame-ancestors (2). Mitigación: agregar X-Frame-Options: DENY o CSP frame-ancestors. – **Media – Spring Actuator Information Leak:** /actuator/health es accesible a través del proxy del frontend (1). Mitigación: no exponer /actuator al público o restringirlo. – **Baja – X-Powered-By:** el servidor revela “X-Powered-By: Express” (3). – **Baja – X-Content-Type-Options:** falta X-Content-Type-Options: nosniff (sistémico).	

EVIDENCIA DE EJECUCIÓN DE PRUEBAS DE: **VULNERABILIDAD**		
	Observación: el escaneo se hizo sobre el servidor de desarrollo; las cabeceras deben agregarse en el servidor de producción y repetir el escaneo.	
Realizado Por: Benjamin Araneda, Vicente Villalobos y Dylan Vitta	**Autorizada Por:** Pendiente – revisión docente	**Estatus:** Marque una Equis [ ] Aprobada **[X] Rechazada** [ ] Diferida

Evidencia de ejecución:

ZAP by Checkmarx **ZAP Report**

Site: http://localhost:4200

Generated on Thu, 22 Oct 2026 14:26:22

ZAP Version: 2.17.0

ZAP by Checkmarx

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	3
Low	2
Informational	2
False Positives:	0

Alerts

Name	Risk Level	Number of Instances
Content Security Policy (CSP) Header Not Set	Medium	3
Missing Anti-clickjacking Header	Medium	2
Spring Actuator Information Leak	Medium	1
Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s)	Low	3
X-Content-Type-Options Header Missing	Low	Systemic
Modern Web Application	Informational	3
User Agent Fuzzer	Informational	Systemic

Figura PV-01. Reporte ZAP del frontend: resumen por riesgo y listado de alertas (zap-report-frontend.html).

EVIDENCIA DE EJECUCIÓN DE PRUEBAS DE: **VULNERABILIDAD**		
ID. Prueba: PV-02 – Escaneo activo OWASP ZAP de las APIs vía OpenAPI (6 microservicios)	**Id. Caso de Uso Asociado:** CU-01, CU-02, CU-03, CU-04, CU-05, CU-07	**Id. Del Requerimiento Asociado:** RNF-03, RNF-05, RNF-09
Fecha de Ejecución: 22/10/2026 15:15	**Herramienta(s) Ocupada(s):** OWASP ZAP 2.17.0 – Automation Framework (zap-plan.yaml): importación OpenAPI, escaneo pasivo y activo	**Alcance:** 142 endpoints de Rol (:8081, 25), Users (:8082, 29), Inventario (:8083, 32), Orden (:8084, 12), Producto (:8085, 29) y Estado (:8086, 15).
Ambiente de Prueba:	Local – Windows 11 Pro. Stack completo levantado con docker-compose: Gateway/BFF (:80), 8 microservicios Spring Boot (Users :8082, Rol :8081, Estado :8086, Inventario :8083, Producto :8085, Orden :8084, Configuración :8087), PostgreSQL y Kafka. Frontend Angular en http://localhost:4200. Contratos importados desde /api-docs de cada servicio.	
Resultado Esperado:	0 alertas Altas o Medias; ante entradas inválidas las APIs deben responder 4xx controlados y no 5xx.	
Resultado Obtenido:	0 Altas, 0 Medias, 3 Bajas y 2 Informativas. Se cumple el criterio principal. Durante el fuzzing se observaron respuestas 5xx en Inventario (58 %), Rol (42 %), Estado (33 %), Users (22 %) y Producto (7 %); Orden no tuvo 5xx.	
Vulnerabilidades Detectadas:	– **Baja – Application Error Disclosure / Debug Error Messages:** HTTP 500 con detalle de error en, por ejemplo, GET /api/roles/, /api/privilegios/, /api/inventario/estantes/, /api/productos/ y /api/estados/ (11 instancias). – **Baja – Timestamp Disclosure:** marcas de tiempo Unix en las respuestas (sistémico). – **Info – Sensitive Information in URL:** correo electrónico en query string: /api/roles/assign?email=… (1). **Recomendación:** manejar las excepciones con un @ControllerAdvice que responda 400/404 sin detalles internos y enviar el correo en el cuerpo del POST. **Nota:** los reportes base individuales por servicio (zap-report-<servicio>.html) no registraron sitios explorados, por lo que no	

EVIDENCIA DE EJECUCIÓN DE PRUEBAS DE: **VULNERABILIDAD**		
	se consideran evidencia. La cobertura de las APIs corresponde a este escaneo OpenAPI.	
Realizado Por: Benjamin Araneda, Vicente Villalobos y Dylan Vitta	**Autorizada Por:** Pendiente – revisión docente	**Estatus:** Marque una Equis **[X] Aprobada** [ ] Rechazada [ ] Diferida

Evidencia de ejecución:

ZAP by Checkmarx **SmartLogix - Escaneo de seguridad de APIs (OpenAPI)**

Escaneo activo de los 6 microservicios Spring Boot via sus contratos OpenAPI

Sites: http://localhost:8086 http://localhost:8085 http://localhost:8084 http://localhost:8083 http://localhost:8082 http://localhost:8081

Generated on Thu, 22 Oct 2026 15:15:23

ZAP Version: 2.17.0

ZAP by Checkmarx

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	0
Low	3
Informational	2
False Positives:	0

Summary of Sequences

For each step: result (Pass/Fail) - risk (of highest alert(s) for the step, if any).

Alerts

Name	Risk Level	Number of Instances
Application Error Disclosure	Low	11
Information Disclosure - Debug Error Messages	Low	11
Timestamp Disclosure - Unix	Low	Systemic
Information Disclosure - Sensitive Information in URL	Informational	1
User Agent Fuzzer	Informational	Systemic

Figura PV-02. Reporte ZAP del escaneo OpenAPI de las APIs (zap-report-apis-openapi.html).

Ejecución de Pruebas de Carga y Estrés

Sistema: SmartLogix

Realizado por: Benjamin Araneda, Vicente Villalobos y Dylan Vitta

EVIDENCIA DE EJECUCIÓN DE PRUEBAS: CARGA Y ESTRÉS		
ID. Prueba: PC-01 – Prueba de carga normal con Apache JMeter (jmeter/load-test.jmx)	**Id. Caso de Uso Asociado:** CU-03 Gestión de Productos (consulta de catálogo)	**Id. Del Requerimiento Asociado:** RNF-01, RNF-02
Fecha de Ejecución: 22/10/2026 19:42–19:44	**Herramienta(s) Ocupada(s):** Apache JMeter 5.6.3 (modo consola, sin GUI) y su dashboard HTML de reportes	**Alcance:** GET /actuator/health y GET /api/productos a través del Gateway (http://localhost:80), con aserción de código HTTP 200 en cada request.
Ambiente de Prueba:	Local – Windows 11 Pro. Stack completo levantado con docker-compose: Gateway/BFF (:80), 8 microservicios Spring Boot (Users :8082, Rol :8081, Estado :8086, Inventario :8083, Producto :8085, Orden :8084, Configuración :8087), PostgreSQL y Kafka. Frontend Angular en http://localhost:4200. Thread Group “Usuarios normales”: 10 hilos, rampa de 30 s, duración 120 s y pausa de 1 s entre iteraciones.	
Resultado Esperado:	Con 10 usuarios concurrentes sostenidos: 0 % de errores y tiempos de respuesta < 2 s (RNF-01); APDEX ≥ 0,94 (umbral de tolerancia 500 ms).	
Resultado Obtenido:	1.045 requests en 2 minutos, 0 errores (0,00 %), 100 % con HTTP 200. Latencia total: promedio 7,23 ms, mediana 7 ms, p95 15 ms, p99 20,54 ms, máximo 37 ms. APDEX 1,000. – **GET /api/productos:** 520 muestras, promedio 10,84 ms, p95 18 ms. – **GET /actuator/health:** 525 muestras, promedio 3,65 ms, p95 6 ms.	
Observaciones:	Bajo carga normal el sistema responde muy por debajo del umbral de RNF-01 y sin errores. El resultado coincide con la prueba equivalente hecha con k6 (PC-02: p95 18,73 ms, 0 % de errores), lo que confirma la medición con dos herramientas distintas.	

EVIDENCIA DE EJECUCIÓN DE PRUEBAS: CARGA Y ESTRÉS

Realizado Por:	**Autorizada Por:**	**Estatus:** Marque una Equis
Benjamin Araneda, Vicente Villalobos y Dylan Vitta	Pendiente – revisión docente	**[X] Aprobada** [ ] Rechazada [ ] Diferida

Evidencia de ejecución:

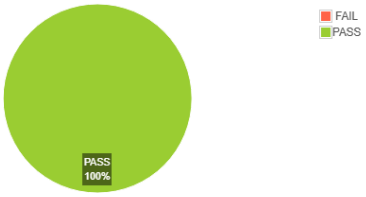
Test and Report information

Source file	"jmeter-load-test-results.jtl"
Start Time	"10/22/26, 7:42 PM"
End Time	"10/22/26, 7:44 PM"
Filter for display	""

APDEX (Application Performance Index)

Apdex	T (Toleration threshold)	F (Frustration threshold)	Label
1.000	**500 ms**	**1 sec 500 ms**	**Total**
1.000	500 ms	1 sec 500 ms	GET /api/productos
1.000	500 ms	1 sec 500 ms	GET /actuator/health

Requests Summary



Statistics

Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label	#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct	99th pct	Transactions/s	Received	Sent
Total	**1045**	**0**	**0.00%**	**7.23**	**2**	**37**	**7.00**	**13.00**	**15.00**	**20.54**	**8.79**	**33.78**	**1.07**
GET /actuator/health	525	0	0.00%	3.65	2	37	3.00	5.00	6.00	9.48	4.43	2.20	0.54
GET /api/productos	520	0	0.00%	10.84	6	28	10.00	15.00	18.00	21.79	4.41	31.86	0.53

Figura PC-01. Dashboard de Apache JMeter de la prueba de carga: APDEX, resumen de requests y estadísticas (jmeter-load-report/index.html).

EVIDENCIA DE EJECUCIÓN DE PRUEBAS: **CARGA Y ESTRÉS**		
ID. Prueba: PE-01 – Prueba de estrés con Apache JMeter (jmeter/stress-test.jmx)	**Id. Caso de Uso Asociado:** CU-03 Gestión de Productos (consulta de catálogo)	**Id. Del Requerimiento Asociado:** RNF-01, RNF-02, RNF-07
Fecha de Ejecución: 22/10/2026 19:44–19:47	**Herramienta(s) Ocupada(s):** Apache JMeter 5.6.3 (modo consola, sin GUI) y su dashboard HTML de reportes	**Alcance:** GET /api/productos a través del Gateway con carga creciente y aserción de HTTP 200.
Ambiente de Prueba:	Local – Windows 11 Pro. Stack completo levantado con docker-compose: Gateway/BFF (:80), 8 microservicios Spring Boot (Users :8082, Rol :8081, Estado :8086, Inventario :8083, Producto :8085, Orden :8084, Configuración :8087), PostgreSQL y Kafka. Frontend Angular en http://localhost:4200. Thread Group "Rampa de estrés": 0 → 150 hilos en 120 s, duración total 150 s y pausa de 0,3 s entre iteraciones.	
Resultado Esperado:	Observar el comportamiento con carga creciente hasta 150 usuarios: tasa de error < 1 % y tiempos < 2 s al menos hasta 50 usuarios (RNF-01); identificar el punto de quiebre si existe.	
Resultado Obtenido:	42.700 requests en 150 s, 0 errores (0,00 %) durante toda la corrida, 100 % con HTTP 200. Throughput promedio 285,29 req/s, con un pico de 489 req/s en un segundo con 150 hilos activos. Latencia: promedio 8,66 ms, mediana 8 ms, p95 17 ms, p99 21 ms, máximo 79 ms. APDEX 1,000.	
Observaciones:	El endpoint de lectura no se degradó ni falló con 150 usuarios concurrentes, por lo que no se encontró el punto de quiebre (misma conclusión que k6 en PE-02). Para encontrarlo hace falta más carga (300–500+ hilos) o un endpoint de escritura (ver PE-03).	
Realizado Por: Benjamin Araneda, Vicente Villalobos y Dylan Vitta	**Autorizada Por:** Pendiente – revisión docente	**Estatus:** Marque una Equis **[X] Aprobada** [ ] Rechazada [ ] Diferida

EVIDENCIA DE EJECUCIÓN DE PRUEBAS: CARGA Y ESTRÉS

Evidencia de ejecución:

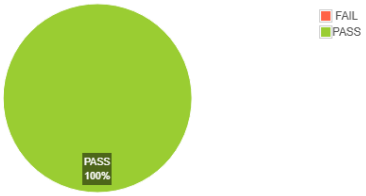
Test and Report information

Source file	"jmeter-stress-test-results.jtl"
Start Time	"10/22/26, 7:44 PM"
End Time	"10/22/26, 7:47 PM"
Filter for display	""

APDEX (Application Performance Index)

Apdex	T (Toleration threshold)	F (Frustration threshold)	Label
1.000	**500 ms**	**1 sec 500 ms**	**Total**
1.000	500 ms	1 sec 500 ms	GET /api/productos

Requests Summary



Statistics

Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label	#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct	99th pct	Transactions/s	Received	Sent
Total	**42700**	**0**	**0.00%**	**8.66**	**3**	**79**	**8.00**	**15.00**	**17.00**	**21.00**	**285.29**	**2059.94**	**34.55**
GET /api/productos	42700	0	0.00%	8.66	3	79	8.00	15.00	17.00	21.00	285.29	2059.94	34.55

Figura PE-01. Dashboard de Apache JMeter de la prueba de estrés: 42.700 requests, 0 % de errores (jmeter-stress-report/index.html).

EVIDENCIA DE EJECUCIÓN DE PRUEBAS: **CARGA Y ESTRÉS**		
ID. Prueba: PC-02 – Prueba de carga normal con k6 (load-test.js)	**Id. Caso de Uso Asociado:** CU-03 Gestión de Productos (consulta de catálogo)	**Id. Del Requerimiento Asociado:** RNF-01, RNF-02
Fecha de Ejecución: 22/10/2026	**Herramienta(s) Ocupada(s):** Grafana k6	**Alcance:** GET /actuator/health y GET /api/productos a través del Gateway (http://localhost:80).
Ambiente de Prueba:	Local – Windows 11 Pro. Stack completo levantado con docker-compose: Gateway/BFF (:80), 8 microservicios Spring Boot (Users :8082, Rol :8081, Estado :8086, Inventario :8083, Producto :8085, Orden :8084, Configuración :8087), PostgreSQL y Kafka. Frontend Angular en http://localhost:4200. Escenario ramping-vus: 0→10 VUs (30 s), 10 VUs sostenidos (1 min), 10→0 (30 s).	
Resultado Esperado:	Umbrales: p(95) de http_req_duration < 500 ms y tasa de error < 1 %. RNF-01: respuestas < 2 s.	
Resultado Obtenido:	1.782 requests (14,77 req/s), 2.673 checks con 100 % de éxito, 0,00 % de errores. Latencia: promedio 6,79 ms, p(95) 18,73 ms, máximo 222,97 ms. Los dos umbrales pasan.	
Observaciones:	Con carga normal el sistema responde muy por debajo del umbral y sin errores (p95 = 18,73 ms). Esta prueba usa 10 usuarios; los 50 usuarios de RNF-01 quedan cubiertos por las pruebas de estrés PE-01 (JMeter) y PE-02 (k6), que llegan a 150. Los resultados coinciden con la prueba de carga de JMeter (PC-01).	
Realizado Por: Benjamin Araneda, Vicente Villalobos y Dylan Vitta	**Autorizada Por:** Pendiente – revisión docente	**Estatus:** Marque una Equis **[X] Aprobada** [ ] Rechazada [ ] Diferida

EVIDENCIA DE EJECUCIÓN DE PRUEBAS: CARGA Y ESTRÉS

Evidencia de ejecución:

```
k6 — Prueba de CARGA (load-test.js)
Target: http://localhost:80 (Gateway)
Escenario: ramping-vus — 0->10 VUs (30s), sostenido 10 VUs (1m), 10->0 VUs (30s)
Endpoints probados: GET /actuator/health, GET /api/productos
Fecha: 2026-10-22


RESULTADOS
  checks_total........: 2673    (100% succeeded, 0% failed)
  http_reqs...........: 1782    14.77/s
  http_req_duration...: avg=6.79ms  min=0.51ms  med=5.78ms  max=222.97ms  p(90)=12.88ms  p(95)=18.73ms
  http_req_failed.....: 0.00%
  iterations..........: 891


THRESHOLDS
  http_req_duration p(95)<500ms .... PASS (18.73ms)
  http_req_failed rate<0.01 ........ PASS (0.00%)
```

Fuente: Docs/evidencia-pruebas/k6-load-test-summary.txt (script k6/load-test.js).

EVIDENCIA DE EJECUCIÓN DE PRUEBAS: **CARGA Y ESTRÉS**		
ID. Prueba: PE-02 – Prueba de estrés de lectura con k6 (stress-test.js)	**Id. Caso de Uso Asociado:** CU-03 Gestión de Productos (consulta de catálogo)	**Id. Del Requerimiento Asociado:** RNF-01, RNF-02, RNF-07
Fecha de Ejecución: 22/10/2026	**Herramienta(s) Ocupada(s):** Grafana k6	**Alcance:** GET /api/productos a través del Gateway con carga creciente.
Ambiente de Prueba:	Local – Windows 11 Pro. Stack completo levantado con docker-compose: Gateway/BFF (:80), 8 microservicios Spring Boot (Users :8082, Rol :8081, Estado :8086, Inventario :8083, Producto :8085, Orden :8084, Configuración :8087), PostgreSQL y Kafka. Frontend Angular en http://localhost:4200. Escenario escalonado 20 → 50 → 100 → 150 VUs (30 s por escalón) y luego 0.	
Resultado Esperado:	Identificar el comportamiento bajo carga creciente; tasa de error < 1 % y respuestas < 2 s hasta al menos 50 usuarios (RNF-01).	
Resultado Obtenido:	31.174 requests (207,49 req/s), 100 % de checks exitosos, 0,00 % de errores con hasta 150 VUs. Latencia: promedio 6,79 ms, p(95) 13,59 ms, máximo 292,68 ms.	
Observaciones:	Sin degradación ni errores en el pico de 150 VUs (207 req/s, p95 = 13,59 ms). La prueba no encontró el punto de quiebre: el endpoint es liviano y el catálogo es pequeño. Se registra como limitación; hace falta más carga (300–500+ VUs) o un endpoint de escritura (ver PE-03). JMeter (PE-01) llegó a la misma conclusión con una herramienta distinta.	
Realizado Por: Benjamin Araneda, Vicente Villalobos y Dylan Vitta	**Autorizada Por:** Pendiente – revisión docente	**Estatus:** Marque una Equis **[X] Aprobada** [ ] Rechazada [ ] Diferida

EVIDENCIA DE EJECUCIÓN DE PRUEBAS: CARGA Y ESTRÉS

Evidencia de ejecución:

```
k6 — Prueba de ESTRES (stress-test.js)
Target: http://localhost:80 (Gateway)
Escenario: ramping-vus progresivo — 20 -> 50 -> 100 -> 150 VUs (30s cada escalon), luego 0
Endpoint probado: GET /api/productos
Fecha: 2026-10-22


RESULTADOS
  checks_total........: 31174   (100% succeeded, 0% failed)
  http_reqs...........: 31174   207.49/s
  http_req_duration...: avg=6.79ms  min=2.42ms  med=5.41ms  max=292.68ms  p(90)=9.76ms  p(95)=13.59ms
  http_req_failed.....: 0.00%
  vus_max.............: 150
```

Fuente: Docs/evidencia-pruebas/k6-stress-test-summary.txt (script k6/stress-test.js).

EVIDENCIA DE EJECUCIÓN DE PRUEBAS: **CARGA Y ESTRÉS**		
ID. Prueba: PE-03 – Prueba de estrés de escritura con k6 (stress-test-write.js)	**Id. Caso de Uso Asociado:** CU-05 Gestión de Órdenes	**Id. Del Requerimiento Asociado:** RF-04, RF-05, RNF-01, RNF-08
Fecha de Ejecución: 22/10/2026	**Herramienta(s) Ocupada(s):** Grafana k6; consulta SQL a PostgreSQL para verificar el stock final	**Alcance:** POST /api/ordenes en el microservicio Orden (:8084): reserva de stock, llamadas a Users y Producto, escritura en PostgreSQL y publicación en Kafka.
Ambiente de Prueba:	Local – Windows 11 Pro. Stack completo levantado con docker-compose: Gateway/BFF (:80), 8 microservicios Spring Boot (Users :8082, Rol :8081, Estado :8086, Inventario :8083, Producto :8085, Orden :8084, Configuración :8087), PostgreSQL y Kafka. Frontend Angular en http://localhost:4200. Escenario 10 → 25 → 50 → 75 VUs (20 s por escalón). Producto de prueba con stock inicial 399.	
Resultado Esperado:	Sin errores 5xx ni timeouts; el stock nunca debe quedar negativo (sin sobreventa) y los pedidos sin stock deben rechazarse de forma controlada.	
Resultado Obtenido:	6.226 requests (62,10 req/s): unos 398 pedidos creados (200) hasta agotar el stock y 5.828 rechazos por stock (400, regla de negocio). 0 errores 5xx y 0 timeouts. Latencia de los 200: p(95) 117,82 ms, máximo 538,85 ms. Stock final del producto: 0 (no negativo).	
Observaciones:	No se encontró un punto de quiebre de infraestructura (0 errores 5xx y 0 timeouts con 75 VUs escribiendo a la vez). Sí se alcanzó el límite de negocio (inventario) y el sistema lo manejó bien: 400 controlado en vez de datos corruptos. La latencia de escritura (p95 = 117,8 ms) es mayor que la de lectura porque cada request llama a otros 2 servicios, escribe en PostgreSQL y publica en Kafka.	

EVIDENCIA DE EJECUCIÓN DE PRUEBAS: **CARGA Y ESTRÉS**		
Realizado Por: Benjamin Araneda, Vicente Villalobos y Dylan Vitta	**Autorizada Por:** Pendiente – revisión docente	**Estatus:** Marque una Equis **[X] Aprobada** [ ] Rechazada [ ] Diferida

Evidencia de ejecución:

```
k6 — Prueba de ESTRES sobre el camino de ESCRITURA (stress-test-write.js)
Target: http://localhost:8084 (Orden, directo, mismo camino que usa el BFF via BffTrustFilter)
Endpoint: POST /api/ordenes (reserva de stock + llamadas a Users/Producto + Kafka + Postgres)
Escenario: ramping-vus — 10 -> 25 -> 50 -> 75 VUs (20s cada escalon), luego 0
Producto usado: "sdvazsa" (id cd2a785e-070e-4ce7-b083-e83acfc2ad64), stock inicial 399
Fecha: 2026-10-22


RESULTADOS
  http_reqs.....: 6226    62.10/s
  Ordenes creadas (200): ~398 (hasta agotar stock)
  Rechazos por stock (400): 5828 (93.6% del total -- rechazo de NEGOCIO, no error de sistema)
  Errores 5xx.....: 0
  Timeouts.....: 0
  http_req_duration (solo 200 exitosos): avg=48.14ms med=24.96ms p(90)=51.33ms p(95)=117.82ms
max=538.85ms


VERIFICACION POST-PRUEBA (contra la base real, no solo la metrica de k6)
  Stock final del producto: 0 (exacto, NO negativo)
  -> Confirma que la reserva atomica de stock (verificacion + descuento en una
     sola operacion, documentada en el informe tecnico) se sostiene bajo
     concurrencia real de 75 VUs simultaneos escribiendo sobre el mismo producto.
     No se encontro overselling / condicion de carrera.
```

Fuente: Docs/evidencia-pruebas/k6-stress-test-write-summary.txt (script k6/stress-test-write.js).

Conclusiones Preliminares del Proceso Realizado

Sistema: SmartLogix

Realizado por: Benjamin Araneda, Vicente Villalobos y Dylan Vitta

CONCLUSIONES PRELIMINARES

INDIVIDUAL – Benjamin Araneda:

Lo que más me dejó este proceso fue ver que una prueba en verde no siempre significa que el sistema funciona. En Katalon, el comando submit marcaba éxito sin iniciar sesión, y al volver a correr las pruebas unitarias aparecieron 3 fallos que los reportes antiguos no mostraban. Ahora reviso que cada prueba tenga una verificación explícita del resultado.

Desde el código, los hallazgos más concretos son el 403 que responde /auth/cambiar-clave ante una clave corta (debería ser 400) y las respuestas 500 que ZAP provocó en las APIs con entradas inválidas. Ambos se resuelven con un manejo de excepciones uniforme (@ControllerAdvice), y es lo primero que corregiría.

INDIVIDUAL – Vicente Villalobos:

Me quedo con la prueba de estrés de escritura: con 75 usuarios creando pedidos al mismo tiempo sobre el mismo producto, el stock terminó exactamente en 0, sin quedar negativo. Eso confirma que la reserva atómica de stock en PostgreSQL funciona bajo concurrencia real, y que el sistema rechaza con 400 los pedidos sin stock en lugar de corromper datos.

En rendimiento cumplimos RNF-01 con amplio margen, y lo confirmamos con dos herramientas distintas: JMeter y k6 coincidieron en 0 % de errores con 150 usuarios y p95 menores a 20 ms en lectura (118 ms escribiendo). Aun así no encontramos el punto de quiebre. Como siguiente paso propongo pruebas con más carga y ejecutar la suite completa de pruebas en un pipeline de CI, para que los fallos se detecten en cada cambio y no al final.

INDIVIDUAL – Dylan Vitta:

Trazar cada prueba a su caso de uso y requerimiento (CU, RF, RNF) permitió saber qué parte del sistema está validada y priorizar los hallazgos. Las 16 pruebas funcionales con Katalon Recorder confirmaron los flujos principales: inicio de sesión, rechazo de credenciales inválidas, cierre de sesión, control de acceso por rol y el ciclo completo de pedidos.

Desde la mirada del usuario, los puntos a mejorar son la seguridad del frontend (faltan cabeceras como CSP y la protección anti-clickjacking) y la accesibilidad: SonarCloud detectó 41 campos de formulario sin etiqueta, lo que afecta a quienes usan lectores de pantalla. Son correcciones simples que mejoran la calidad percibida del sistema.

CONCLUSIONES GENERALES OBTENIDAS (GRUPALES):

Como equipo aplicamos a SmartLogix pruebas funcionales automatizadas, unitarias, de integración, análisis estático, análisis de vulnerabilidades y pruebas de carga y estrés. Esto nos dio una visión completa de la calidad del sistema. Estas son nuestras conclusiones:

Pruebas automatizadas: con Katalon Recorder validamos de punta a punta la autenticación, la sesión, el control de acceso por rol, el carrito y el ciclo de pedidos: 16 casos automatizados, todos aprobados. Lo más valioso fue detectar falsos positivos en los scripts iniciales: comandos que quedaban en verde sin probar nada (submit sobre un formulario Angular, sesión que persistía entre casos). Aprendimos que una prueba automatizada sin verificaciones explícitas no aporta evidencia.

Pruebas unitarias y de integración: al volver a ejecutar las 681 pruebas unitarias de los 8 microservicios, 678 pasaron y 3 fallaron. Dos son pruebas desactualizadas frente al código actual; la tercera reveló un defecto real: /auth/cambiar-clave responde 403 en vez de 400 ante una clave corta. Los reportes anteriores en verde no garantizaban el estado actual del código. La prueba de integración de Orden pasó (4/4) y confirmó que la creación de pedidos funciona con la seguridad, el servicio y la base de datos reales.

Análisis estático: SonarCloud mostró 0 bugs en el backend y mantenibilidad A. La calificación de seguridad D se debe sobre todo a hallazgos que hay que revisar en contexto: CSRF deshabilitado en APIs stateless con JWT es una decisión válida, mientras que los endpoints sin método HTTP explícito sí deben corregirse. El frontend aprobó el Quality Gate, pero tiene 48 bugs de accesibilidad (campos sin etiqueta).

Seguridad: ZAP no encontró riesgos Altos. Sí encontró 3 riesgos Medios en el frontend (falta de CSP y de protección anti-clickjacking, y /actuator expuesto) y respuestas 500 en las APIs ante entradas inválidas, lo que indica que falta un manejo de errores uniforme.

Rendimiento: medimos con Apache JMeter y con k6, y ambas herramientas llegaron al mismo resultado: 0 % de errores con hasta 150 usuarios concurrentes (JMeter: 42.700 requests, pico de 489 req/s, p95 17 ms). El sistema cumplió RNF-01 con amplio margen, incluso escribiendo con 75 usuarios simultáneos (p95 118 ms), y la reserva atómica de stock evitó la sobreventa. Queda pendiente encontrar el punto de quiebre real con más carga.

Oportunidades de mejora: (1) agregar cabeceras de seguridad HTTP en el frontend y restringir /actuator; (2) manejar las excepciones en las APIs para no responder 500, y corregir el 403 de /auth/cambiar-clave; (3) actualizar las 3 pruebas unitarias que fallan y ejecutar la suite completa en cada cambio (idealmente en CI); (4) corregir la accesibilidad del frontend; (5) revisar y cerrar en SonarCloud los hallazgos de seguridad; (6) ampliar las pruebas de estrés.

Cada hallazgo quedó trazado a su caso de uso y requerimiento, lo que nos permite priorizar las correcciones y repetir las mismas pruebas como regresión después de corregirlas.